



Penetration Testing

PoolParty 2nd (dApp)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The PoolParty team partnered with Hashlock to conduct a security Penetration Test of their PoolParty Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

The PoolParty™ Exchange & Marketplace is positioning itself as an innovative decentralized trading platform (DEX) that aims to eliminate the common cost of "price impact" in crypto trading. By integrating an over-the-counter (OTC) marketplace directly into the DEX structure, PoolParty™ enables sellers—especially large holders—to execute trades without affecting market prices, resulting in higher profits for them and attractive discounts for buyers. The platform is styled around the tagline "Let the chart fly", underscoring its goal of seamless, impact-free crypto trading.

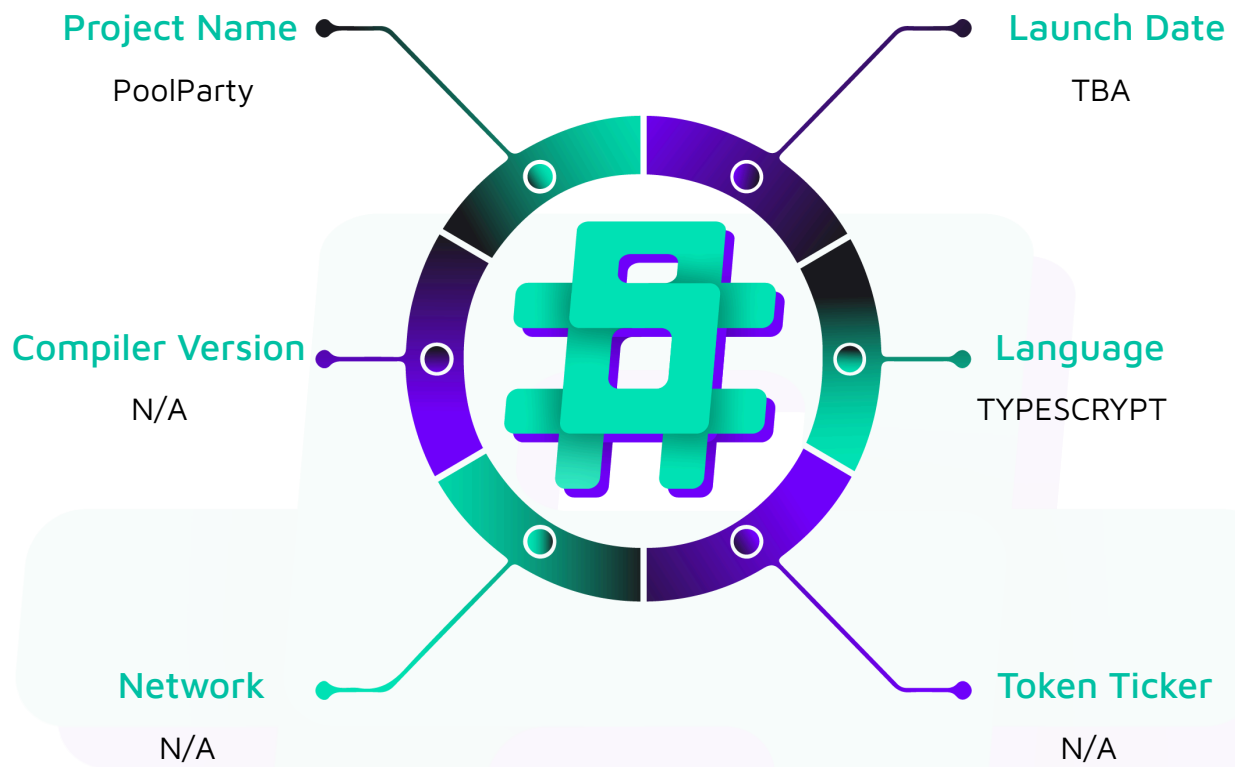
Project Name: PoolParty

Project Type: dApp

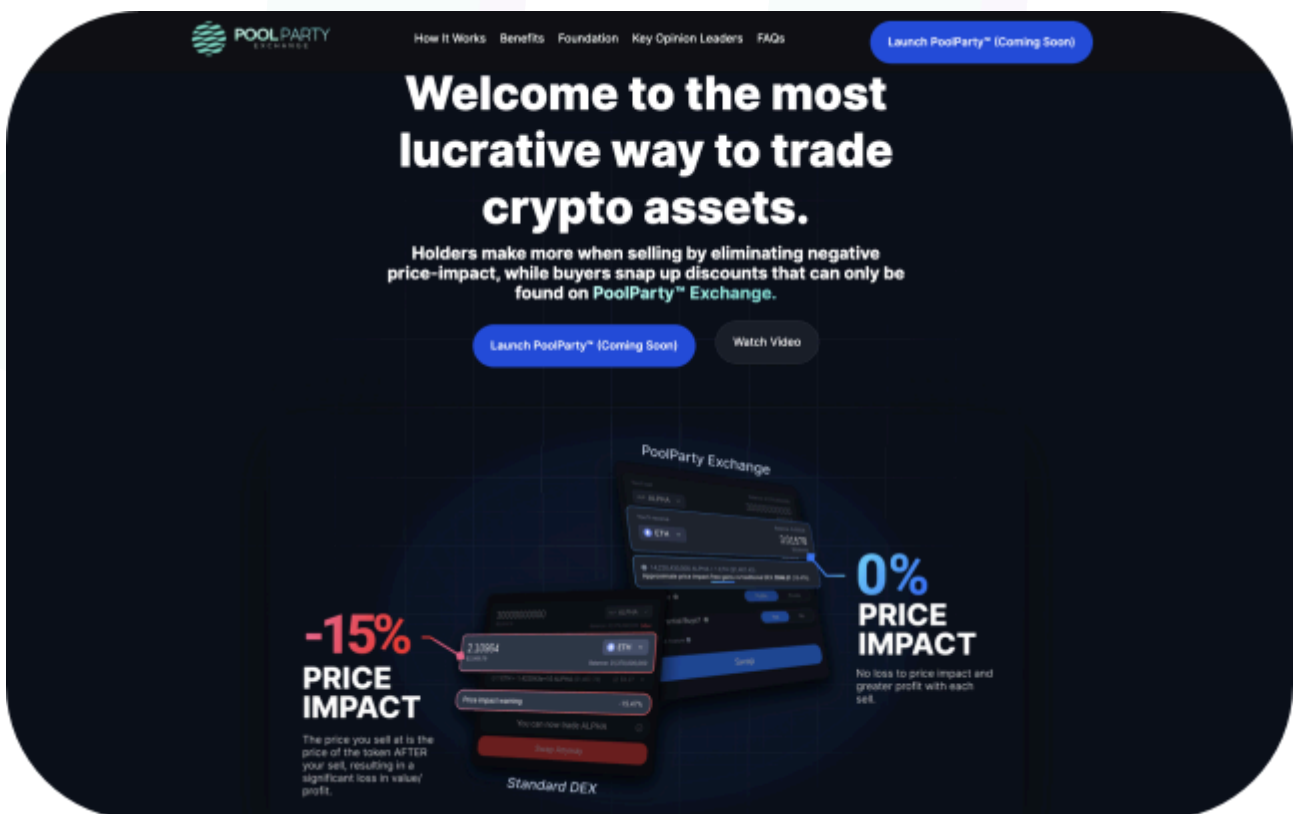
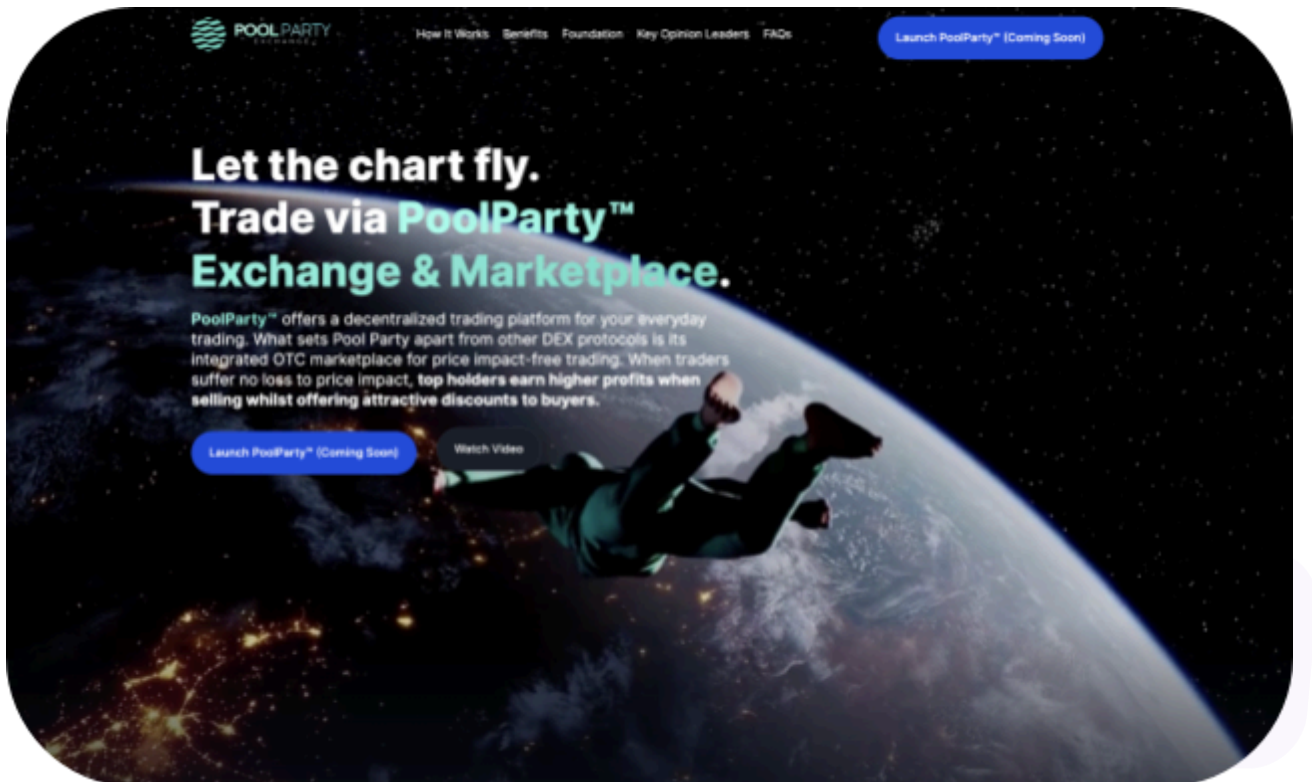
Website: <https://www.poolparty.market/>

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the PoolParty project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	PoolParty Project Code Base
Platform	Ethereum / Solidity/Web apps & APIs
Penetration Test Date	August, 2025
Repository/Website URL/IP Tested	https://github.com/PoolParty-Exchange/poolparty-app
GitHub Commit Hash (If applicable)	fe6771e05d2e57639e397bcc002af6d7129447c1
Fix Commit Hash (If applicable)	3267fc9185268f33d951958791a9abe03b09cf73

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed, and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

4 High severity vulnerabilities

2 Medium severity vulnerabilities

1 Low severity vulnerability

Caution: Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] `src/pages/api/smartaccount/getpaymaster#getpaymaster` - Credential Exposure of Paymaster Keys

Description

The `getpaymaster` endpoint returns the `paymasterKey` directly to clients based on a client-supplied chain value, exposing sensitive infrastructure credentials used to sponsor transactions.

Vulnerability Details

In `src/pages/api/smartaccount/getpaymaster.ts`, the handler parses `BICONOMY_PAYMASTERS` from `process.env` and responds with the selected `paymasterKey` to any authenticated request using only lightweight token format checks. An attacker can call this endpoint with a validly formatted (but unverified) token and extract the `paymaster` key for a target chain.

Proof of Concept:

- 1) Prepare a syntactically valid JWT (any three-part string as described in the previous issue).
- 2) Send a GET request to `/api/smartaccount/getpaymaster` with header: `chain: <targetChainId>` and `Authorization: Bearer <synthetic-token>`.
- 3) The API responds with `{ paymasterKey }`, revealing the key.

Impact

Exposing `paymaster` keys allows malicious actors to relay or submit sponsored transactions at your expense, potentially draining budgets, causing denial of service for legitimate users, and creating reputational and financial damage. Once leaked, rotating and redistributing new keys across all dependent services is operationally complex.



Recommendation

`paymasterKey` keys must never be exposed client-side. Refactor the architecture to keep keys strictly on the server. Instead of sending the key, the client should request the server to sponsor a specific, validated transaction. The server-side endpoint must:

1. Fully authenticate the user by cryptographically verifying their JWT.
2. Validate that the user is authorized to perform the requested transaction.
3. Use the paymaster key to sign and relay the transaction on the user's behalf.

Status

Resolved

[H-02] src/utls/apiUtils - Per-user Rate Limiting Bypass via Unverified JWT Claims

Description

The API derives user-specific rate-limiting keys from the sub-claim of an unverified JWT. Since the token is not cryptographically verified, an attacker can forge arbitrary subjects and rotate them per request, effectively bypassing per-user rate limits.

Vulnerability Details

In `src/pages/api/profile/getuser.ts`:

- `validatePrivyToken` only checks that the Authorization header looks like a JWT (three segments), without signature verification.
- `getIdFromToken` base64-decodes the JWT payload without verifying the signature and reads `payload.sub`.
- `checkRateLimit(req, 200, 60000, `user:${userId}`)` uses this attacker-controlled `userId` to key the limiter.

Proof of Concept:

- 1) Craft any three-part token with a chosen subject:

Header: `{"alg":"none","typ":"JWT"}` → `eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0`

Payload: `{"sub":"attacker-001"}` → `eyJzdWliOiJhdHRhY2tldCIowMDEifQ`

Signature: arbitrary → `eA`

Token: `eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWliOiJhdHRhY2tldCIowMDEifQ.eA`

- 2) Send repeated requests, rotating sub (`attacker-001`, `-002`, `-003`, ...). Each request is treated as a different user for the limiter: ``user:attacker-XXX``.
- 3) The per-user rate limit is nullified as the attacker can generate unlimited buckets.

Impact

The per-user rate-limiting controls are rendered ineffective. This allows an attacker to send a high volume of requests from a single source, which can be used to launch denial-of-service attacks, degrade service for legitimate users, and drive up operational costs. It also makes it significantly more difficult to trace and block malicious actors.

Recommendation

Enforce cryptographic verification of all JWTs before their claims are used for any security-sensitive function, including rate limiting. The `userId` for the rate limiter's key should only be extracted *after* the token's signature, issuer, and audience have been validated. If full verification cannot be performed before rate limiting, key the pre-authentication limit to a more stable, non-forgable property, such as a hash of the raw `Authorization` token string combined with the source IP address.

Status

Resolved

[H-03] `src/pages/api/profile/updateuser` - Broken Object Level Authorization in Profile Updates

Description

The `/api/profile/updateuser` endpoint forwards the entire client-provided request body to an upstream service without first verifying that the authenticated user is authorized to perform the requested changes. This could allow an attacker to forge requests to modify profile data belonging to other users.

Vulnerability Details

In `src/pages/api/profile/updateuser.ts`, the handler forwards the `req.body` directly to the upstream API. The endpoint fails to perform a critical authorization check: ensuring the user ID in the verified token matches the user ID being targeted for modification in the request body. Compounded by the unverified JWT issue, an attacker can specify any target user in the body and have the server process the request.

Impact

If the downstream service also fails to perform strict ownership checks, this vulnerability allows for unauthorized modification of any user's data. This can lead to account takeover, data theft, and further propagation of attacks within the system. It fundamentally breaks the trust boundary between users.

Recommendation

Implement and enforce strict, server-side authorization checks. After cryptographically verifying the JWT, extract the authenticated user's ID from its claims. Before processing any update, ensure that the resource being modified belongs to this authenticated user. Do not trust the request body to specify the target of the operation; use the verified identity from the token as the sole source of truth for the user's identity.

Status

Resolved

[H-04] `src/pages/api/profile/getuser` - Insecure Direct Object Reference (IDOR) via Client-Controlled Headers

Description

The `/api/profile/getuser` endpoint accepts a `pid` (presumably a user or profile identifier) directly from a request header and uses it to retrieve user data. Because the header value is fully controlled by the client, an attacker can manipulate it to attempt to access data belonging to other users, leading to information disclosure.

Vulnerability Details

In `src/pages/api/profile/getuser.ts`, the line `const pid = req.headers.pid as string;` reads the identifier directly from the header. This `pid` is then used to construct the request URL for the downstream service. An attacker can simply iterate through different `pid` values in the header of their requests to systematically query for other users' profiles.

Impact

This vulnerability can lead to a mass breach of user data. An attacker could potentially enumerate and exfiltrate sensitive profile information for all users of the application, leading to a major privacy incident, loss of user trust, and regulatory penalties.

Recommendation

Do not use client-controlled headers to specify the target of a data-retrieval operation. The identity of the user whose data is being requested should be determined exclusively from a cryptographically verified JWT. The endpoint should be redesigned to fetch data only for the authenticated user associated with the validated token, ignoring any other identifiers passed in headers or the request body.

Status

Resolved

Medium

[M-01] Public Exposure of RPC and Service Keys

Description

The application embeds sensitive RPC endpoint URLs containing API keys directly into the client-side code. Specifically, keys for the `drpc.org` service are exposed, making them publicly accessible to anyone inspecting the application's network traffic or JavaScript files.

The application is configured to use RPC endpoints from environment variables prefixed with `NEXT\_PUBLIC\_` (e.g., `NEXT\_PUBLIC\_DEFAULT\_ETHER\_RPC` in `src/lib/web3.ts`). These variables resolve to URLs for services like `drpc.org` that include an authentication key, as demonstrated by the following client-side request:

```
POST    /ogrpc?network=ethereum&dkey=Au8pyt9a0UK3gC19svsGJI6Vt4w485YR74TL-gTye0yN
HTTP/2

Host: lb.drpc.org

Content-Type: application/json

Origin: https://pp-remake.vercel.app

{"jsonrpc": "2.0", "id": 26, "method": "eth_call", ...}
```

The `dkey` parameter in the URL is the publicly exposed API key. An attacker can easily extract this key by observing the application's network requests.

Impact

If the exposed keys belong to a paid, usage-based RPC service, an attacker can abuse them by sending a high volume of requests. This can lead to significant and unexpected financial costs by exhausting service quotas. Once a quota is exceeded, the service will be disabled for all legitimate users, causing a denial-of-service (DoS) condition that impacts application availability and reliability until the next billing cycle.



Recommendation

Never expose private API keys or full RPC URLs with embedded keys on the client side. Instead, implement a backend proxy or API gateway that securely stores these keys and forwards requests to the appropriate service.

1. Move all environment variables containing sensitive keys (e.g., the `dkey` for `drpc.org`) from `NEXT_PUBLIC_` to server-side-only variables.
2. Create a new serverless function or API endpoint (e.g., `/api/rpc-proxy`) that receives requests from the client.
3. This server-side endpoint should append the secret key to the request and forward it to the actual RPC provider.
4. Implement robust rate-limiting and authentication on this proxy endpoint to prevent abuse.

This approach ensures that keys remain secret on the server while still allowing the client application to access the necessary services through a controlled, secure channel.

Status

Resolved

[M-02] Lack of Hardening: Permissive Image Configuration & Missing CSP

Description

The application's security posture is weakened by two configuration issues: the Next.js image loader allows images to be sourced from any external hostname (`'hostname: ""'`), and no Content Security Policy (CSP) is implemented. While not direct vulnerabilities, these omissions remove critical layers of defense against common web attacks.

Impact

The overly permissive image policy increases the application's exposure to cross-site scripting (XSS) via malicious SVGs or other image-based attacks and can be abused for user tracking or phishing. The absence of a CSP means that if an XSS flaw were to be discovered elsewhere, there would be no browser-level defense to prevent an attacker from executing arbitrary scripts, stealing session cookies, or exfiltrating data.

Recommendation

Adopt a principle of least privilege for security configurations.

1. ****Image Remote Patterns:**** Restrict the `'images.remotePatterns'` in `'next.config.ts'` to a specific allowlist of known, trusted hostnames where application images are stored.
2. ****Content Security Policy:**** Implement a strong CSP, preferably via middleware. Start with a restrictive policy (`'default-src 'self''`) and explicitly allow only the origins necessary for application functionality (e.g., for scripts, styles, fonts, and API connections).

Status

Resolved

Low

[L-01] HTTP Security Header Misconfiguration

Description

It was observed that the application has misconfigured HTTP security headers, including `Access-Control-Allow-Origin` and `Cross-Origin-Resource-Policy`. Additionally, it lacks crucial security headers such as `X-Frame-Options`, `Content-Security-Policy`, and `Strict-Transport-Security`. These headers are essential for protecting against various web security vulnerabilities and ensuring secure communication between the website and users.

Impact

Misconfigured security headers can expose a website to various attacks, including Cross-Site Request Forgery (CSRF) and data breaches due to improper `Access-Control-Allow-Origin` and `Cross-Origin-Resource-Policy` settings. Weak `Content-Security-Policy` configurations can lead to Cross-Site Scripting (XSS) and data injection attacks, allowing malicious scripts to steal user data or deface the website. The absence of `X-Frame-Options` makes the site vulnerable to clickjacking, where attackers can trick users into unintended actions. Without `Strict-Transport-Security`, man-in-the-middle attacks become a threat, enabling attackers to intercept sensitive communications. Additionally, headers like `X-Powered-By` and `ETag` can lead to information disclosure, providing attackers with insights into the website's technology stack or exposing sensitive metadata.

Recommendation

It is recommended to implement and configure the `Access-Control-Allow-Origin` and `Cross-Origin-Resource-Policy` headers. Implement the missing HTTP security headers (`X-Frame-Options`, `Content-Security-Policy`, and `Strict-Transport-Security`) with appropriate directives to mitigate.

Status

Resolved



Conclusion

After Hashlock's analysis, the PoolParty project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.